

**Московский авиационный институт
(национальный исследовательский университет)**

Кафедра 806 «Вычислительная математика и программирование»

Объектно-ориентированное программирование (семестр 1)

01.03.02 «Прикладная математика и информатика»

ЛАБОРАТОРНАЯ РАБОТА 1: ОСНОВЫ C++ — ОДНА ПРОГРАММА НА ДИНАМИЧЕСКИХ СТРУКТУРАХ ДАННЫХ

Общая информация

Параметр	Значение
Срок сдачи	2 недели после выдачи
Prerequisite	Лекции 1–4 (типы, операторы, указатели и ссылки, функции, CMake)
Язык	C++ 20
Система сборки	CMake 3.16+
Тестирование	Google Test

Цели

По итогам работы вы напишете **одну законченную консольную программу** — интерактивный справочник, который работает с динамической структурой данных вашего варианта (динамический массив, C-строка или матрица) и выполняет над ней набор операций, выбираемых из текстового меню. Вся логика оформляется как **библиотека чистых функций**: функции не вводят и не выводят данные сами, а принимают их параметрами, изменяют переданные структуры (в том числе по ссылке через *out-параметры*) и возвращают результат. Работа состоит из двух частей:

1. **Часть 1** — единая программа варианта: динамическая структура данных (массив, C-строка или матрица), базовые операции семейства, алгоритм варианта и поиск с возвратом результата *и* найденного значения по ссылке.

Вся логика — чистые функции (указатели, ссылки, out-параметры), собранные в один проект с текстовым меню.

2. **Часть 2** — проект CMake с Google Test, который проверяет чистые функции программы.

Важно. Запрещены классы, конструкторы, семантика копирования/перемещения, `std::string`, `std::vector`, `std::array` и умные указатели. Работа с памятью — только вручную (`new[]` / `delete[]`), строки — только C-строки (`char*`, завершающий `'\0'`).

Часть 1. Единая программа варианта

Основное задание. Ваш вариант закреплён за одним из трёх **семейств** структур данных (см. таблицу вариантов). Вы пишете **одну** программу `main.cpp` с текстовым меню, которая работает со структурой своего семейства: создаёт её, выводит, выполняет базовые операции, алгоритм варианта и поиск.

Общая схема программы

Бесконечный цикл с пунктами меню (номера и формулировки уточнены в семействе):

1. Создать структуру данных
2. Напечатать
3. Операция семейства
4. Алгоритм варианта
0. Выход

Обязательства программы (одинаковы для всех вариантов):

- **Никаких утечек памяти:** каждая созданная через `new[]` структура освобождается через `delete[]`. После удаления указатель обнуляется (`delete[] arr; arr = nullptr;`).
- **Корректный ввод:** при вводе чисел и меню проверяйте успешность `std::cin` (`if (!(std::cin >> n)) { std::cin.clear(); ... }`), не давайте программе упасть.
- **Устойчивость к пустой структуре:** функции поиска и алгоритмы должны корректно работать с массивом длины 0 (возвращать `false`/безопасное значение), программа не должна падать.
- Каждая операция выводит понятный результат (сам массив, длину, найденные значения).

Семейство А — линейный динамический массив (варианты 1–8)

Базовые операции в `array_ops.h` / `array_ops.cpp`:

```
// array_ops.h
#pragma once
#include <cstdint>

int* array_create(std::size_t size);           // new int[size], +1 слот под '\0'
не нужен
void array_delete(int*& arr);                 // delete[] и обнулить указатель
int* array_resize(int* arr, std::size_t size, std::size_t new_size); // новая память
+ копия
int* array_insert(int* arr, std::size_t& size, std::size_t pos, int value); // вставка
int* array_remove(int* arr, std::size_t& size, std::size_t pos); // удаление
void array_print(const int* arr, std::size_t size);
// Ваш вариант задания, например:
bool array_binary_search(const int* arr, std::size_t size, int target, std::size_t&
    out_index);
```

Меню семейства А:

- | | |
|----------------------|--------------------|
| 1. Создать массив | 2. Напечатать |
| 3. Вставить элемент | 4. Удалить элемент |
| 5. Изменить размер | 6. Печать |
| 7. Алгоритм варианта | 0. Выход |

Семейство В — С-строки через указатели char* (варианты 9–14)

Базовые операции в str_ops.h / str_ops.cpp:

```
// str_ops.h
#pragma once

char* str_alloc(const char* src);             // копия строки в новой динамической памяти
std::size_t str_len(const char* s);          // длина до '\0'
void str_copy(char* dst, const char* src);    // копирование в существующий буфер
void str_delete(char*& s);                   // delete[] и обнулить
void str_print(const char* s);
```

Меню семейства В:

- | | |
|----------------------|-------------------------------------|
| 1. Ввести строку | 2. Напечатать |
| 3. Длина | 4. Скопировать в буфер и напечатать |
| 5. Алгоритм варианта | 0. Выход |

Помните: под строку всегда выделяется strlen + 1 байт — один слот под завершающий '\0'.

Семейство С — двумерные массивы, матрицы int** (варианты 15–20)

Базовые операции в matrix_ops.h / matrix_ops.cpp:

```
// matrix_ops.h
#pragma once
#include <cstdint>
```

```
int** matrix_create(std::size_t rows, std::size_t cols); // массив указателей на строки
void matrix_delete(int** m, std::size_t rows);           // каждая строка, затем массив строк
void matrix_fill(int** m, std::size_t rows, std::size_t cols, int value);
void matrix_print(const int* const* m, std::size_t rows, std::size_t cols);
```

Меню семейства C:

- | | |
|--------------------|------------------------|
| 1. Создать матрицу | 2. Заполнить значением |
| 3. Напечатать | 4. Алгоритм варианта |
| 0. Выход | |

Матрица — это массив указателей на строки: `m = new int*[rows]`, каждая строка — `m[i] = new int[cols]`. Удаление — в обратном порядке.

Часть 2. CMake и Google Test

Не менее **10 тестов** Google Test.

Тесты `test_variant.cpp` (пример для семейства A): проверьте создание/удаление массива, вставку в середину, удаление, бинарный поиск, а затем алгоритм варианта на контрольном примере.

Варианты индивидуальных заданий

Чистые функции (указатели, ссылки, out-параметры) и часть 2 (CMake, Google Test) одинаковы для всех вариантов.

Основное задание (часть 1) — единая консольная программа, работающая с динамической структурой данных, которую задаёт семейство варианта:

- **A** — линейный динамический массив (1–8),
- **B** — C-строки через указатели `char*` (9–14),
- **C** — двумерные массивы, матрицы `int**` (15–20).

Базовые операции семейства приведены в части 1, алгоритм и дополнительные функции — в таблице, подробные требования к каждому варианту — ниже. При сдаче укажите номер варианта.

№	Семейство	Сортировка / поиск	Обязательные функции	Алгоритм
1	А · Массивы	Пузырьковая + бинарный поиск	<code>size_t array_unique(int* arr, size_t size)</code>	$O(n^2)$ обмены; удаление дубликатов in-place
2	А · Массивы	Выбором + бинарный поиск	<code>void array_reverse(int* arr, size_t size)</code>	Минимум в хвосте; разворот двумя указателями
3	А · Массивы	Вставками + бинарный поиск	<code>void array_rotate_left(int* arr, size_t size, size_t k)</code>	Вставка в префикс; циклический сдвиг влево
4	А · Массивы	Gnome-сортировка + бинарный поиск	<code>size_t array_remove_all(int* arr, size_t& size, int value)</code>	Сравнение соседей; удаление всех вхождений
5	А · Массивы	Подсчётом (0...1023)	<code>size_t array_range_count(const int* arr, size_t size, int lo, int hi)</code>	Таблица частот; подсчёт в [lo, hi]
6	А · Массивы	Слиянием (рекурсия)	<code>int array_median(const int* arr, size_t size)</code>	Медиана за $O(n \log n)$
7	А · Массивы	Быстрая (рекурсия)	<code>int array_kth_smallest(int* arr, size_t size, size_t k)</code>	Разбиение Хоара, k-й по величине элемент
8	А · Массивы	Слияние двух отсортированных	<code>int* array_merge(const int* a, size_t na, const int* b, size_t nb, size_t& out_size)</code>	Два указателя, один проход
9	В · С-строки	—	<code>void str_reverse(char* s) + bool str_is_palindrome(const char* s)</code>	Два указателя с краёв к центру
10	В · С-строки	—	<code>size_t str_count_words(const char* s) + bool str_find_substr(const char* s, const char* sub, int& out_pos)</code>	Переходы пробел→буква; наивный поиск
11	В · С-строки	—	<code>void str_replace_char(char* s, char from, char to) + size_t str_remove_char(char* s, char ch)</code>	Замена; сжатие сдвигом влево
12	В · С-строки	—	<code>void str_to_upper(char* s) + size_t str_count_char(const char* s, char ch)</code>	Регистр ASCII; счётчик символов
13	В · С-строки	—	<code>char* str_substr(const char* s, size_t pos, size_t len) + int str_compare(const char* a, const char* b)</code>	Копия подстроки; лексикографическое сравнение
14	В · С-строки	—	<code>char* str_trim(const char* s) + void str_reverse_words(char* s)</code>	Убрать пробелы с краёв; разворот порядка слов
15	С · Матрицы	—	<code>int** matrix_transpose(const int* const* m, size_t rows, size_t cols)</code>	$m \times n \rightarrow n \times m$

№	Семейство	Сортировка / поиск	Обязательные функции	Алгоритм
16	C · Матрицы	—	void matrix_row_swap(int** m, size_t cols, size_t r1, size_t r2) + void matrix_col_swap(int** m, size_t rows, size_t c1, size_t c2)	Обмен указателей на строки / элементов столбца
17	C · Матрицы	—	int* matrix_row_max(const int* const* m, size_t rows, size_t cols) + int* matrix_col_min(const int* const* m, size_t rows, size_t cols)	Максимумы строк / минимумы столбцов
18	C · Матрицы	—	int** matrix_mul(const int* const* a, size_t rows_a, size_t cols_a, const int* const* b, size_t rows_b, size_t cols_b)	Произведение матриц $A(n \times m) \cdot B(m \times k)$
19	C · Матрицы	—	int* matrix_spiral_read(const int* const* m, size_t rows, size_t cols, size_t& out_size)	Обход по спирали в одномерный массив
20	C · Матрицы	—	bool matrix_is_symmetric(const int* const* m, size_t n) + int matrix_trace(const int* const* m, size_t n)	$a[i][j] == a[j][i]$; сумма главной диагонали

Задания вариантов (подробно)

Вариант 1. А · Массивы. Сортировка пузырьком по возрастанию; затем `size_t array_unique(int* arr, size_t size)` — удалить все дубликаты на месте (сдвигом влево), вернуть новую длину. Программа выводит массив после сортировки и после удаления дубликатов.

Пример: {1, 3, 3, 5, 1, 3} → после сортировки {1, 1, 3, 3, 3, 5} → после `array_unique` {1, 3, 5}, длина 3.

Вариант 2. А · Массивы. Сортировка выбором по возрастанию; затем `void array_reverse(int* arr, size_t size)` — развернуть массив на месте двумя указателями. Вывод после сортировки и после разворота.

Пример: {3, 1, 4, 1, 5} → {1, 1, 3, 4, 5} → {5, 4, 3, 1, 1}.

Вариант 3. А · Массивы. Сортировка вставками по возрастанию; затем `void array_rotate_left(int* arr, size_t size, size_t k)` — циклический сдвиг влево на k (например, разворотом трёх отрезков).

Пример: массив {4, 2, 5, 1, 3} → сортировка {1, 2, 3, 4, 5} → сдвиг на k=2 → {3, 4, 5, 1, 2}.

Вариант 4. А · Массивы. Gnome-сортировка по возрастанию; затем `size_t array_remove_all(int* arr, size_t& size, int value)` — удалить все вхождения `value` сдвигом влево; `size` обновляется по ссылке, возвращается число удалённых.

Пример: {2, 5, 2, 8, 2, 1}, `value=2` → {5, 8, 1}, `size` станет 3, вернуть 3.

Вариант 5. А · Массивы. Сортировка подсчётом (значения в диапазоне 0...1023); затем `size_t array_range_count(const int* arr, size_t size, int lo, int hi)` — количество элементов в отрезке [`lo`, `hi`].

Пример: {3, 7, 3, 9, 5, 2}, [3, 6] → 3.

Вариант 6. А · Массивы. Сортировка слиянием (рекурсия); затем `int array_median(const int* arr, size_t size)` — медиана: элемент с индексом `size/2` в отсортированном массиве (для чётного размера — левый из двух средних).

Пример: {5, 3, 1, 4, 2} → 3.

Вариант 7. А · Массивы. Быстрая сортировка (рекурсия, разбиение Хоара); затем `int array_kth_smallest(int* arr, size_t size, size_t k)` — `k`-й по величине элемент (нумерация с 1).

Пример: {4, 2, 7, 1, 9, 3}, `k=3` → 3.

Вариант 8. А · Массивы. Программа вводит два отсортированных массива `a` и `b` и строит их слияние: `int* array_merge(const int* a, size_t na, const int* b, size_t nb, size_t& out_size)` — один проход двумя указателями, размер результата — `out_size` параметр.

Пример: `a={1, 3, 5}`, `b={2, 4, 6}` → {1, 2, 3, 4, 5, 6}, `out_size=6`.

Вариант 9. В · С-строки. `void str_reverse(char* s)` — развернуть строку на месте двумя указателями; `bool str_is_palindrome(const char* s)` — проверить, является ли строка палиндромом. Программа выводит развёрнутую строку и результат проверки.

Пример: `level` → разворот `level`, палиндром `true`; `hello` → `olleh`, `false`.

Вариант 10. В · С-строки. `size_t str_count_words(const char* s)` — число слов (подсчёт переходов «пробел → не пробел»); `bool str_find_substr(const char* s, const char* sub, int& out_pos)` — `true`, если подстрока найдена, позиция записывается в `out_pos` по ссылке.

Пример: `hello world test` → 3 слова; `world` → `true`, `out_pos=6`; `xyz` → `false`.

Вариант 11. В · С-строки. `void str_replace_char(char* s, char from, char to)` — заменить все `from` на `to`; `size_t str_remove_char(char* s, char ch)` — удалить все `ch` сдвигом влево, вернуть число удалённых.

Пример: banana, a→o → bonono; удаление n → baаа, удалено 2.

Вариант 12. В · С-строки. `void str_to_upper(char* s)` — перевести латинские буквы в верхний регистр (ASCII); `size_t str_count_char(const char* s, char ch)` — сколько раз встречается символ.

Пример: Hello, Mai → HELLO, MAI; count 'l' → 2.

Вариант 13. В · С-строки. `char* str_substr(const char* s, size_t pos, size_t len)` — выделить подстроку в новой динамической памяти (с завершающим '\0'); `int str_compare(const char* a, const char* b)` — лексикографическое сравнение (результат <0, 0 или >0).

Пример: hello, `str_substr(s, 1, 3)` → ell; `compare("abc", "abd")` <0; `compare("abc", "abc")` =0.

Вариант 14. В · С-строки. `char* str_trim(const char* s)` — вернуть новую строку без пробельных символов по краям; `void str_reverse_words(char* s)` — развернуть порядок слов (слова разделены одним пробелом), сами слова не разворачивать.

Пример: " hi " → hi; one two three → three two one.

Вариант 15. С · Матрицы. `int** matrix_transpose(const int* const* m, size_t rows, size_t cols)` — построить матрицу cols×rows, где `result[j][i] = m[i][j]`; исходную не менять.

Пример: [[1,2],[3,4],[5,6]] (3×2) → [[1,3,5],[2,4,6]] (2×3).

Вариант 16. С · Матрицы. `void matrix_row_swap(int** m, size_t cols, size_t r1, size_t r2)` — обменять две строки, поменяв местами указатели; `void matrix_col_swap(int** m, size_t rows, size_t c1, size_t c2)` — обменять элементы двух столбцов.

Пример: в [[1,2],[3,4],[5,6]] обмен строк 0 и 2 → [[5,6],[3,4],[1,2]].

Вариант 17. С · Матрицы. `int* matrix_row_max(const int* const* m, size_t rows, size_t cols)` — массив максимумов строк; `int* matrix_col_min(const int* const* m, size_t rows, size_t cols)` — массив минимумов столбцов.

Пример: [[3,1,4],[5,2,0]] → `row_max` = {4, 5}, `col_min` = {3, 1, 0}.

Вариант 18. С · Матрицы. `int** matrix_mul(const int* const* a, size_t rows_a, size_t cols_a, const int* const* b, size_t rows_b, size_t cols_b)` — произведение матриц; при `cols_a != rows_b` вывести сообщение об ошибке.

Пример: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$.

Вариант 19. С · Матрицы. `int* matrix_spiral_read(const int* const* m, size_t rows, size_t cols, size_t& out_size)` — обход матрицы по спирали (слева направо, затем сверху вниз и т. д.) в одномерный массив; размер результата — out-параметр.

Пример: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightarrow \{1, 2, 3, 6, 9, 8, 7, 4, 5\}$.

Вариант 20. С · Матрицы. `bool matrix_is_symmetric(const int* const* m, size_t n)` — верно, если `m[i][j] == m[j][i]` для всех пар; `int matrix_trace(const int* const* m, size_t n)` — сумма главной диагонали.

Пример: $\begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$ — симметрична, след = 4.